**Proceedings of**

**, , ,**

# IMPORTING TERACACHE TO THE JAVA PARALLEL OLD GARBAGE COLLECTOR (CS527 PROJECT)

**Emmanouil Anagnostakis**
MSc Student
Department of Computer Science
University of Crete
Email: manosanag@csd.uoc.gr

## ABSTRACT

*Managed analytics frameworks cache intermediate results in memory (on-heap) or storage devices (off-heap) to avoid costly recomputations, especially in graph processing.As datasets grow, on-heap caching requires more memory for long-lived objects, resulting in high garbage collection (GC) overhead. On the other hand, off-heap caching moves cached objects on the storage device, reducing GC overhead, but at the cost of serialization and deserialization (S/D). These problems are avoided by TeraCache,an existing work of an on heap compute cache based on a memory mapped heap that is not garbage collected. TeraCache eliminates both S/D and GC over cached data without imposing any language restrictions.*

*TeraCache is implemented in the HotSpot JDK 8 Parallel Scavenge GC in the Serial Old implementation of the major collection. Serial Old is now a deprecated old generation collector and Parallel GC accepts only the Parallel Old collector for the old generation. In this work I make an attempt to import TeraCache to the Parallel Old collector and evaluate it with some Java microbenchmarks, because a small bug remains and i didn't feel comfortable to move on with Spark benchmarks (i.e. Spark bench, Renaissance). Parallel Old TeraCache shows a maximum 44% performance improvement against Serial Old TeraCache on the best real case scenario created with microbenchmarks. These microbenchmarks are Java tests than run with a significant number of Java objects and collect the heap frequently. An extra feature called Parallel TeraCache is implemented to help boosting the performance of Tera compaction specifically. While not showing much of an improvement it is still better than having a single threaded TeraCache compaction.*

## INTRODUCTION

Big data analytics frameworks, such as Spark are designed to analyze and extract meaningful insights from huge volumes of data. Typically, such processing requires iterative computations over data until a convergence condition is met. Each iteration produces new transformations of previously computed data, and this process generates a massive volume of intermediate objects. Previous work and our measurements show that it is important to cache intermediate results due to the significantly higher cost of recomputation. Caching intermediate results tends to be costly as their cumulative volume is often several times larger than the input dataset and they exhibit long lifetimes, spanning several iterations of long computations. Caching intermediate results on the heap creates significant problems. As intermediate data grows in size, the pressure on the managed heap grows as well, resulting in frequent garbage collection (GC) cycles with low yield as each cycle reclaims little space. Similar behaviors occur in other application domains, e.g. key-value stores that maintain large amounts of cached data and metadata in memory for data serving. Therefore, the common practice to cope with growing datasets and high GC cost is to move objects outside the managed heap (off-heap). Off-heap caching can happen in memory (DRAM) as well, although typically off-heap caches use a fast storage device. In addition, the state of the art practice, e.g., in Spark, is to use a combination of on-heap and off-heap caching. Unfortunately, moving managed objects off-heap requires expensive serialization/deserialization (S/D). The TeraCache evaluation shows that GC and S/D overheads often constitute the larger part of the execution time in applications that handle large datasets. More importantly, off-heap caching lim-

its the objects that can be cached, as it requires self-contained entities.

TeraCache needs to run on a more modern and faster garbage collector than the Serial Old collector. By many researches it is clear that Parallel Scavenge in general is the go to collector for big data analytics frameworks because of the high throughtput it provides in comparison with other collector e.g.the default G1. The default old generation collector for the young collector Parallel Scavenge (scavenge meaning young collection) is the Parallel Old collector. This work manages to import TeraCache to the Parallel Old collector and achieve 44% performance improvement against the existing version of TeraCache working with the Serial Old. Overall my work makes the following contributions: 1) Injecting the TeraCache code to Parallel Old by minimizing every possible overhead 2) A parallel TeraCache compaction feature with the collector deploying separate threads than its own to make the migrations from the Java Heap to the Tera Heap.

## BACKGROUND

In this section a detailed background of the garbage collectors TeraCache runs on will be given to be easier to follow the rest of the report. Both collectors aim to sweep the old generation of the collected heap of its garbage and run along with the Parallel Scavenge collector which scavenges the young generation. Before we start I want to clear out two specific terms. A minor collection is a collection of the young generation. A major collection is a collection of the old generation. A full collection is a collection of both the young and the old generation. Major collections never happen on their own but always as a full collection in order to bring objects from the young generation to be collected in the major collection as well. Moreover when doing a major collection one more minor collection is happening at the same time for the pointers of the moved tenured objects pointing to young objects to be adjusted. Parallel Scavenge divides the young generation into 3 spaces, Eden and the Survival spaces S0 and S1. When a mutator thread allocates an object in our application that object is put in the eden space. Minor collections are trigerred when the eden space is full. The survival spaces work as swapping meaning that always one contains objects and the other is an empty destinaton for the other semi space and the eden space. In the next minor collection, all the objects from eden will be moved to the empty semi space along with some objects that survived in the other survival space. Objects that survived many gc cycles (default 15) in the survival spaces are promoted to the old generation.

### Serial Old Collector

The Serial Old Collector is a major collector that works on par with all Java minor garbage collectors except with G1. It is a single threaded major collection code divided in four phases. The collector's phases are visualized in figure 1.

The first phase is called the marking phase. In marking phase the collector identifies all object references following some well known root pointers that are placed in the stack. These objects are marked as live objects and are the objects that will survive the current major gc and be compacted in the start of the old space to avoid fragmentation of the dead objects in order to free space for new objects to be allocated in the old space. The collector uses a stack called a marking stack to put an object whenever it marks it. After following the closure of an object and marking all its children the collector has pushed all these objects in the marking stack. When finished with all these children objects he will pop the first child from the marking stack to follow it leading in a depth first search traversal of the object graph. When the collector is done following all the object graphs i.e. all roots it will move to the next phase.

The second phase is called the precompaction phase.In this phase the collector scans the old space and assigns the new addresses to each object where they will be moved during the fourth phase. The new address is called the forwarding pointer and is stored inside the mark work of each object. The mark word is a 32 bit word that holds 30 bits for object forwarding pointer and 2 bits to indicate its state. If the last 2 bits are both set the object is marked. The collector scans the whole old generation until it encounters the allocation top pointer which is set to the last live object in the heap. The new addresses start from the beginning of the old generation to cover any possible fragments on dead objects.

The third phase is called the adjust phase. In this phase the collector scans the entire old space and follows each object to indentify all of its children and adjust the pointers containing their address to point to their new addreses. Before scaning the whole heap the collector adjusts the pointers of the root objects which as we said when describing the marking phase are residing in the stack and the root object is inside the heap. When the collector finishes adjusting all the live objects it is ready to move the objects to their new addresses.

The fourth phase is the phase where the gc compacts all the objects to the beginning of the old generation. It scans the old space and for every live object is finds it retrieves its forwarding pointer. It then performs a memmove operation and moves the object to its new address and resets its mark word for the next collection. It will keep doing the same thing for all objects it encounters in the space. After finishing all the migrations the allocation top is put after the last compacted object for the new objects that will come from the young generation or directly allocated in the old generation.

**Parallel Old Collector** The Parallel Old Collector is a major collector that works on par with the Parallel GC which is a minor garbage collector. It is a multithreaded major collection

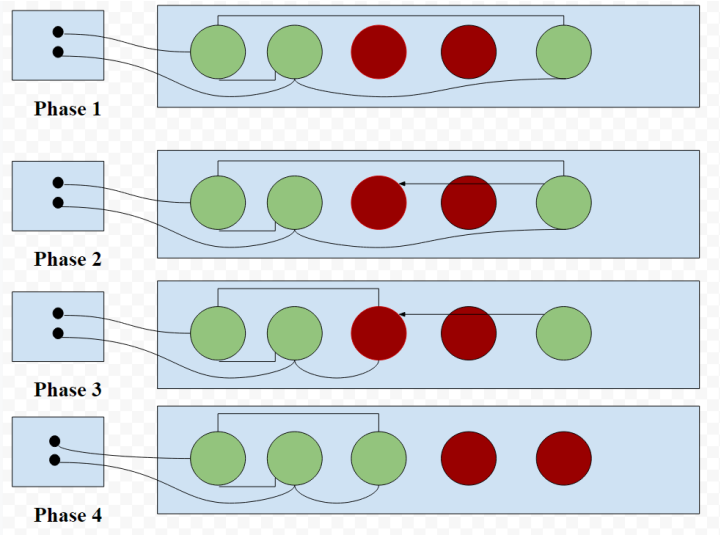


**FIGURE 1**. The phases of Serial Old GC. Live objects are colored as green and dead objects as red.

code divided in four phases. The collector's phases are visualized in figure 2.

The first phase is called the marking phase and it is multithreaded. It is exactly the same as the marking phase of the serial old collector. The difference is that it is multithreaded and tasks should be created in order for gc threads to take them and execute them.There is also one more critical difference as the collector here uses a bitmap to keep track of marked objects and not a mark word for each object but this works in a very similar way. For the task part there is a task queue where the tasks are enqueued. Each object graph beginning from its root is given its own gc task and is put in the queue. Every thread will dequeue a task that is in the queue and start executing it in a core. If there are more threads than the root tasks a thread will be given a steal marking task to steal work from the other threads so as not to remain idle. Moreover each thread has its own compaction manager. There is also the master thread called the vm thread which will stop running until the workers have completed all the tasks in the task queue. The vm thread also has its own compaction manager but does no work. Every compaction manager contains the marking stack for its corresponding thread so each thread marks and pushes to each own marking stack in parallel to complete marking its object graph in the way described in the serial old collector section.

The second phase is called the summary phase. It is a single threaded phase as it is hard to the forwarding kind of work in parallel. It is different than the precompaction phase of the serial old collector but does work of the same quality. In the start of the collection before the marking phase the space is divided into regions so it can be scavenged in parallel by the worker threads. These regions are filled in the marking phase. When an object is identified and marked it is added to its region. So now comes the summary phase to compute the destination address not for each object but for each region of objects. This phase works as a preparation phase for the compact phase which is the last.

The third phase is the adjust roots phase. Here only the roots ,not the object closure, are adjusted by a single thread. As mentioned the root pointers reside in the stack while the root objects reside in the heap and are adjusted to point to the root objects' new addresses in their destination region.

The fourth phase is the compaction phase. It is a multithreaded code part where move-and-update tasks are created and enqueued to the task queue. Each moved-and-update task is responsible for compacting a region of objects which is called a move-and-update closure by executing a memmove operation to its new address in the destination region and also to update its pointer closure by following its children. The forwarding pointer is stored in the mark bitmap section of each object. Some objects are going to reside in the start of each region and these are called the dense prefix of the region as these are objects that need not be moved. These objects belong to an update-only closure and only their closure pointers are being updated while not being moved. When all move-and-update tasks have been completed by the threads, the vm thread will update the closure pointers of the deferred objects of each region. These objects are objects that are cut in half by a region boundary and need special care for that reason meaning that they are updated last. However the partial objects will be moved during the multithreaded section of this phase with all the other objects of the space. After the compaction phase the mark bitmap is reset for the next collection.

## DESIGN

In this section we are going to follow the full path the author has followed to make TeraCache work for the parallel old collector. It is divided in four parts from the first compilation to assigning TeraCache its own threads to run on.

### Setting up the running environment

TeraCache is a huge work with a large code investment. It needs three steps to be taken in order for TeraCache to run. These steps are about the three components that TeraCache uses, a region allocator, a fast storage device and the Java Virtual Machine OpenJDK 8. As a beginning we need to compile the region allocator which is built as a seperate component. TeraCache is divided into regions and in each region reside objects of the same closure. For Spark the object closure is called a partition of an RDD. The size of each region is 512 MB. The region allocator is responsible for allocating new objects in the TeraCache regions. When compiling the region allocator a .so library file is created that is then linked with the rest of JVM library files. The sec-

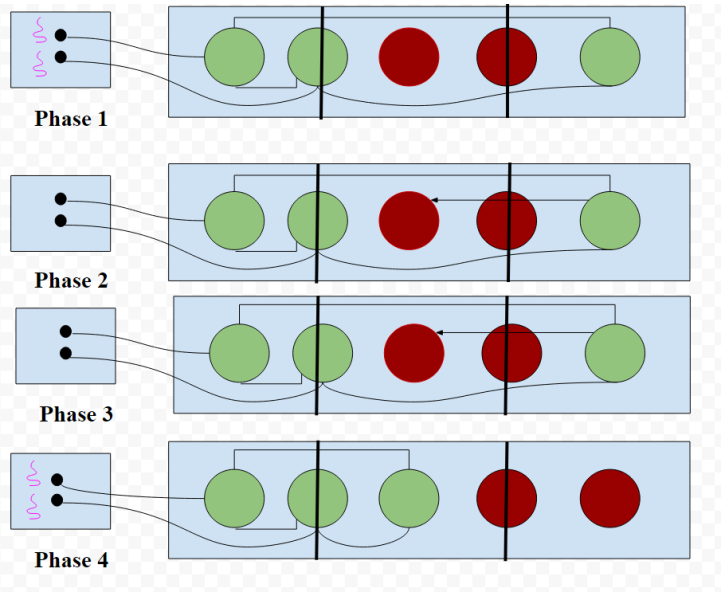


**FIGURE 2**. The phases of Parallel Old GC. Live objects are colored as green and dead objects as red. Threads are pink.

ond step we need to take is to mount a fast storage device. In our computer we have an NVMe storage device which we use in order to memory map the Tera Heap. We create a file to be used for the TeraCache reservations of the NVMe storage for its purposes. The last step that needs to be taken is the configuration and the compilation of the JVM. We will talk about the configuration in the Evaluation section below. After the configuration, we compile the JVM with a simple make command and when the operation is finished we are ready to run with TeraCache. TeraCache with Parallel Old is depicted in figure 3.

### Injecting the TeraCache code to Parallel Old

After setting up the running environment I needed to study the Parallel Old code to understand how it works and the phases it is divided into. The most important part was to understand how the gc threads work as parallelism is the main subject we are focused on in terms of this project. I splitted the code reading into 4 segments one regarding each phase of the collection. The marking phase is very similar to that of the Serial Old collector. It consists of the following parts. The first part is creating gc tasks to enqueue in the gc task queue. I will not dive into details as this was done in the Background section. After the task are put to the queue the execution starts. Each thread will dequeue a task from the queue and start executing them. It will start by marking the root and pushing it to its own marking stack. Then it is going to pop the root object from the start to follow its class object and identify all its referenced children objects. Every referenced children object will be marked and pushed to the marking stack. In terms of TeraCache there is an extra function being imlemented that marks the object to be moved to TeraCache. The tera flag is the mark word of TeraCache. It consists of 64 bits that hold information about its region, its new address and its state. When an object closure is marked by the application to be moved to TeraCache then this object will be marked for the heap entering the normal mark-and-push function while the rest of the closure is going to be marked using the extra function called tera-mark-and-push.When an object is tera marked it will be put to a vector to be processed in seperate by the other sections of the collection and be moved to Tera Heap. To continue with the class object, in every class object class there is a function for the serial old collector and one for the parallel old collector. So what need to be done was to copy the code from Serial old functions to the Parallel Old ones with caution to call the right mark-and-push or tera-mark-and-push function of the Parallel Old code. After finishing with this code I moved to the main marking phase function and copied all the code from the Serial Old marking phase function taking care for the main gc function as well. The only difference in the marking phase code is that before the root marking needs to occure the marking of all the forward pointers to the Tera Heap. This marking part needs to call the mark-and-push function of the Parallel Old collector. When finished with the marking phase i moved to the second phase called the summary phase which is single threaded. This was the hardest phase to understand as it calculates the destination adress of each region. It scans all the heap starting from the old space to calculate if it fits to itself. If it does not fit to itself it has a second destination space that comes from the virtual space which is the reserved space of memory for the Java Heap. After calculating the addresses of all old regions it does the same for the young generation because we need to compact the young generation as well to adjust its pointers that were changed while moving the old generation objects. After the summary phase is the adjust roots phase. The adjust roots phase is single threaded and with one pass adjust all the root pointers residing in the stack. The code that needs to be inserted here is a segment that adjusts all the forward pointers i.e. the objects that point from the heap to TeraCache. This part is copied from the adjust phase of Serial Old with the difference that it needs to call the Parallel Old adjust pointer function. This function checks if in a object belongs to TeraCache or not and if it does that pointers remain to the address in TeraCache, if not it extracts its forwarding pointer from the bitmap to update its references. The last phase is the compaction phase. This phase is multithreaded and for each region it compacts it moves its objects to the destination region and also follows their closure to update its pointers. I do not dive in much detail because i explained it in the Background section. In addition because this phase will dive into the class objects to find the objects referents we need to move TeraCache code in these classes. Each update-contents function in the classes is identical to the one of Serial Old called adjust-pointers. What we need to

do is copy the TeraCache code from the latter to the former.

### Distinguishing the bitmap from the mark word

The most critical step of this work was to understand that the objects moving to TeraCache must not be visible to the heap and because the objects were added to the regions and not scanned from the heap directly that distinction was to be done in the marking phase. So the outcome was that the bitmap should not be used because its usage is hard and heap bound. The obvious choice was the Serial Old mark word as the vessel of our forwarding pointer. When an object is marked for the Tera Heap it is pushed to a vector and marked in the mark word. Same happens for objects that are already tera flaged by the applicattions and are going to be marked in the ordinary mark-and-push. After the marking phase the objects put to this vector are given an address in a TeraCache region. What I do is fetch the current allocation top of the region this object should be moved to and store it in the mark word. Afterwards in the adjust phase what i do is before following the roots I scan the vector to follow each TeraCache object's children and adjust their pointers to their new addresses stored in the mark word or in the bitmap if the object is a heap object. Then in the compaction phase I copy all the objects contained in the vector to the Tera Heap adresses calculated in the forwarding pointers of the mark word. After the heap compact i make sure that all tera flags are reset in the heap.

### Scaling with more than one threads

In order to scale with more than one gc threads what I needed to understand is that there must be a vector for each compaction manager i.e. for each thread. These vectors are filled in the marking phase and because double objects might occur inside them as the threads are easily racing on the same object some measures must be taken to know that these objects are already processed. This procedure is a part of the precompaction phase of TeraCache where an object that has a forwarding pointer already pointing to an adress in TeraCache is not assigned an address in TeraCache again. When a forwarding pointer is being calculated the object is put to another single vector so as to avoid the multiple vectors. This vector is then sequentially scanned in the third phase and all the children of tera objects are followed for their pointers to be adjusted. Finally it is rescanned in the beginning of the compaction phase so that the objects will move to their new addresses in TeraCache. This is done before the heap compaction so as to avoid to overwrite objects that need to move to TeraCache. The vectors that are used are then cleared when the heap compaction is ended.

### Parallel TeraCache

What i noticed is that the marking of TeraCache is done in parallel as it is done identically with the heap by the gc threads. The summary phase of TeraCache could not be parallelized because we need many allocation pointers for the regions, one for each thread otherwise the threads would race for same address. However parallelizing the adjust and compaction phase was not a difficult task as these phases do not have something to worry about thread racing. The Parallel TeraCache approach is simple. We divide the Tera vector into exactly as many parts as the threads we will be using. I chose to use pthreads as I am familiar with using this library and because managing gc threads is not an easy task. I use two vectors one to store the starting index of each vector part and one to store its ending index. Each thread will then fetch its own index and adjust and compact the vector part sequentially. Because the vector size modulo the number of threads might not be 0, the small amount of remaining objects is then adjusted and compacted by the vm thread. The threads are synchronized using a pthread barrier so the vm thread will wait and not continue executing the rest of the code.

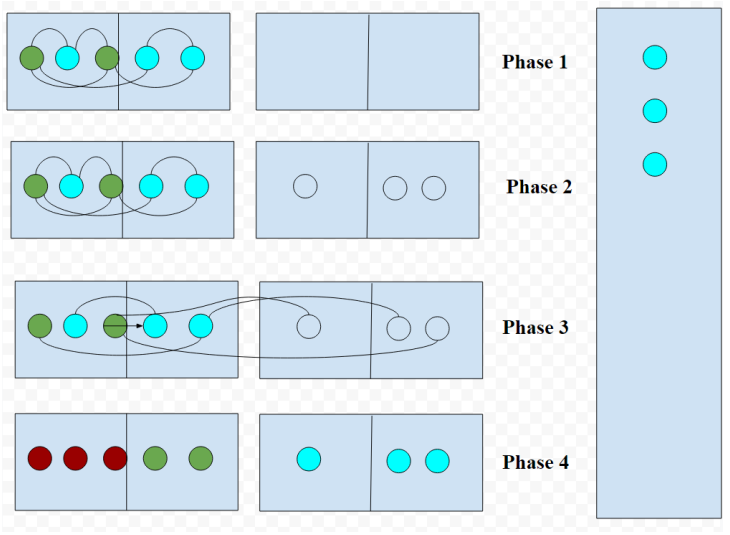


**FIGURE 3**. The phases of Parallel Old GC. Live objects are colored as green, dead objects as red and tera objects are blue.

## EVALUATION

In this section the author will provide details about the computer setup and configurations used. Moreover there will be given a detailed analysis on the benchmarks used to evaluate our work

### Setup

I deployed the JVM on a 32-core computer to have fast compilations and execution times because OpenJDK is a project that

needs a lot of processing power. The Java version is Java 8. I reserved 100g space on an NVMe device and used 5.4gb for the Tera Heap which were memory mapped to avoid serialization/deserialization overhead. JVM was configured without debug symbols, which were not needed, with a GCC 4.8.5 compiler and 32 compilation threads. For the compilation of the Java tests I used the Java Compiller that comes in hand with the configuration of the JVM. Regarding the Java Heap I used a scaling heap starting on 15mb and being able to scale to 20gb. I used uncompressed oops which i support as a good pratice and i scaled my applications from 1 to 16 threads. My implementation was not able to scale to more than 24-28 threads so i chose to remain to 16 at this point. In addition i created a new JVM flag to be able to specify the number of TeraCache threads to test the Parallel TeraCache implementation. I also tried scaling parallel TeraCache with 1 to 16 Tera threads.

### Benchmarks

Before beginning implementing TeraCache for Parallel Old i wished that i could deploy it on Spark in the end. A bug is still reproducable so i chose not to go with Spark but remain with the Java microbenchmarks containg collected ArrayLists/LinkedLists of like 30m objects. These benchmarks were created to reproduce bugs and simulate real time applications with a lot of collections. They contain Java data structures like ArrayList and LinkedList. The usage of data structures made testing TeraCache easier because TeraCache has a built-in feature that with a Java annotation you can mark a whole closure to be moved to the Tera Heap. These microbenchmarks were centerred to be able to test some certain charecteristics of applications than run with TeraCache. There might be scenarios when the most objects will be in the heap i.e. belong to Spark's execution memory and others when most objects will be in RDDs i.e. belong to Spark's storage memory which in our case is TeraCache. The most realistic scenario is that like 2/3 of the objects will reside insdie the heap as short-lived objects and 1/3 of the objects will be cached in Tera Heap's RDDs. Bellow follows brief a analysis of the Java data structures used in these scenarios to have a better understanding of their inner implementation and how it affects our application's performance.

An ArrayList is a data structure that represents a dynamic array. In Java, Arrays are fixed size structures of data residing in the heap in continuous space. When someone wants to add more objects to the Array than the specified size he simply cannot. That is where ArrayList comes into play being an expandable Array with an initial size of 10. When the programmer wants to to add the 11th element a new Array of size 20 is created and all objects are copied inside.By looking at how this structure works we notice that it does not need any reference except one to the starting block of the ArrayList. The accesses are pretty fast because pointer arithmetic is easy when you work with continuous space. However adding an element to the ArrayList when you need to resize it has a cost of O(N) meaning you need to iterate all over the current Array to resize it. This results to the fast that with a lot of objects a lot of resizes must happen leading to more time complexity in terms of additions.

A LinkedList is a list of elements that each one points to the next element.When the programmer adds a new element to the list, it is done pretty quickly by adjusting the pointers of the head of the list. However that does not mean that it has a perfect locality, because the objects that are in continous space in the heap are only those objects that are inserted in the list one after the other. If you add an object later on it can be residing anywhere in the heap even in another generation. So adding an object is pretty quick with 0(1) time complexity. But retrieving a certain object is not done by indexing, as with the continous Array elements, but by following the next pointer of each object until it reaches the desired one. Having to follow so many pointers means extra gc time, because the main thing gc does is following pointers. So we expect to witness the more dramatic improvement in gc time when running these benchmarks with the parallel old collector compared to the serial old one.

## RESULTS

In this section i will present all the results retrieved from running the benchmarks described in the previous section in all of the different scenarios.

### Parallel Old Vanilla vs Serial Old Vanilla

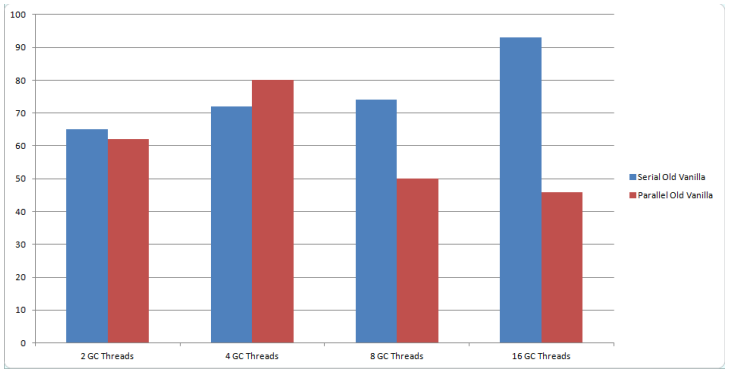


**FIGURE 4**. Parallel Old Vanilla vs Serial Old Vanilla performance with 2, 4, 8 and 16 gc threads respectively. Major GC time is depicted in seconds in x axis. (ZOOM RECOMMENDED.)

Figure 4 shows performance of both Parallel and Serial Old garbage collectors when they have to collect objects with a lot of references such as a LinkedList with 30 million objects. It is

clear that when running with 2 or 4 threads the overhead added by creating the threads and their tasks is hiding any serious performance benefit but with 8 and 16 threads maximum thoughput on pointer processing is achieved hence the great performance on the side of parallel. As threads increase serial old gc is affected in a performance degrading way as only extra overhead is added to major gc cause of breaking inner gc locality from the minor collection who is executed in parallel.

### Parallel Old Vanilla vs Parallel Old TC

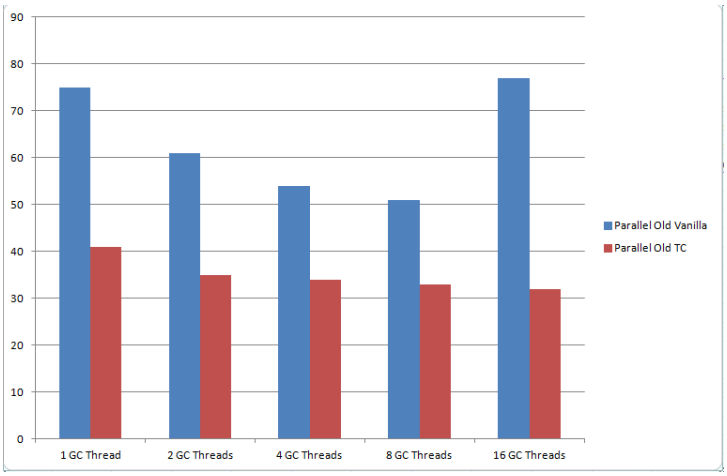


**FIGURE 5**. Parallel Old Vanilla vs Parallel Old TC performance with 2, 4, 8 and 16 gc threads respectively. Major GC time is depicted in seconds in x axis. (ZOOM RECOMMENDED.)

Figure 5 shows performance of Parallel Old when objects reside only in the heap and when objects are moved to TeraCache and the heap is freed from garbage collection. The application is faster with differences scaling from 15 to 50 seconds. It is normal for vanilla to go faster when max thread utilization reaches its limit with 4 and 8 threads because a lot of objects that create traffic inside the heap are processed in parallel. We notice that with 16 threads vanilla hits a high thread overhead because the threads do not have much work to do and remain idle. Parallel Old TC times are not improved very much by scaling the number of gc threads because there is no traffic with heap objects who are processed in parallel. In contrast TeraCache runs with a steady 16 TC Threads.

### Parallel Old Vanilla vs Serial Old TC and everything moves to TeraCache

Figure 6 shows performance of Parallel Old TC vs Serial Old TC. What we understand from this graph is that if everything moves to TeraCache there is no direct benefit from gc threads

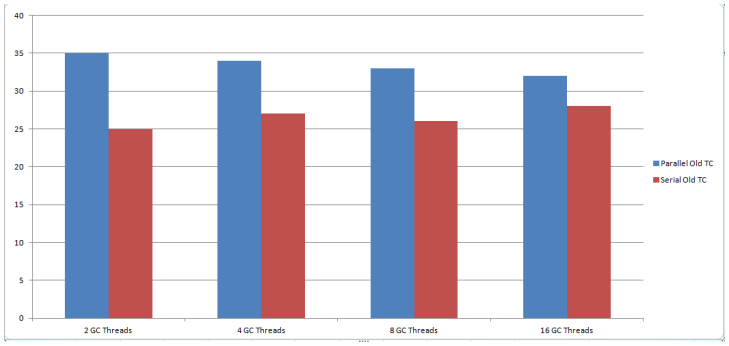


**FIGURE 6**. Parallel Old TC vs Serial Old TC performance with 2, 4, 8 and 16 gc threads respectively in an example that everything moves to TeraCache. Major GC time is depicted in seconds in x axis. (ZOOM RECOMMENDED.)

since the heap has no traffic and we are doomed to pay the gc thread overhead without improving any performance. Parallel Old GC is useful in real scenarios when a portion of the heap is moved to TeraCache and not every single object.

### Parallel TC feature scaling

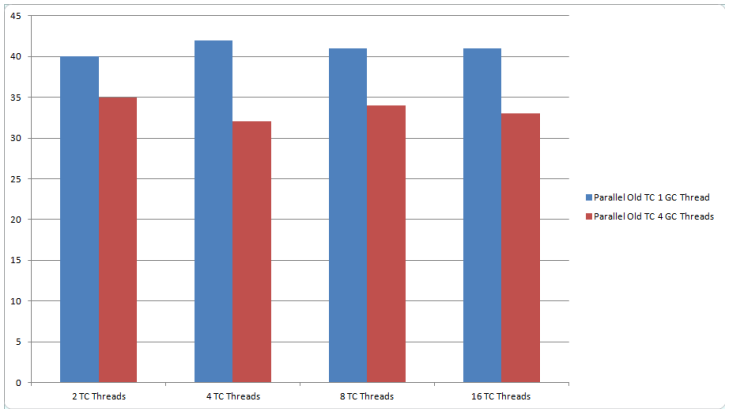


**FIGURE 7**. Parallel TC with 1 gc thread vs Parallel TC with 4 gc threads performance with 2, 4, 8 and 16 tc threads respectively in an example that everything moves to TeraCache. Major GC time is depicted in seconds in x axis. (ZOOM RECOMMENDED.)

Figure 7 shows performance of the Parallel TeraCache feature scaling to 2, 4, 6, 8 and 16 tc threads respectively with comparing 1 and 4 gc threads as well.

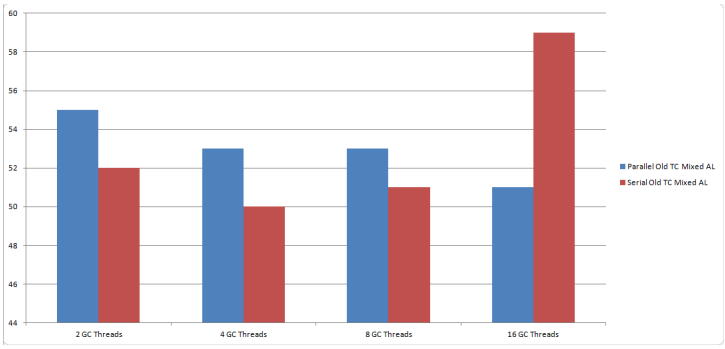


**FIGURE 8**. Parallel Old TC vs Serial Old TC performance with 2, 4, 8 and 16 gc threads respectively in an example that 2/3 of objects remain in heap and 1/3 is moved to Tera Heap in the end showing no weight reduction inside the heap. Major GC time is depicted in seconds in x axis. (ZOOM RECOMMENDED.)

### Parallel Old Vanilla vs Serial Old TC in more real time scenarios

Figure 8 shows that when we dont have many floating objects in the heap like in an ArrayList and a single pass to the ObjArrayKlass object is enough to indentify all the contents with indexing, a low level of parallelism does not overcome the thread overhead which is huge. This means that a single thread processes the whole ArrayList. Only when scaling to 16 threads we are able to get a positive performance with Parallel Old.

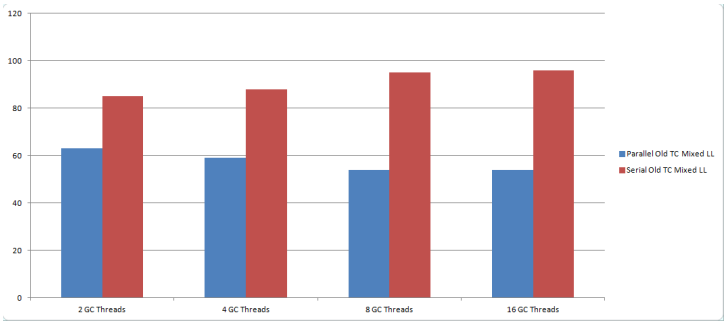


**FIGURE 9**. Parallel Old TC vs Serial Old TC performance with 2, 4, 8 and 16 gc threads respectively in an example that 2/3 of objects remain in heap and 1/3 is moved to Tera Heap in the end showing no weight reduction inside the heap. Major GC time is depicted in seconds in x axis. (ZOOM RECOMMENDED.)

Figure 9 shows that when we have many pointers between our heap objects like in a LinkedList and the objects can be found from any root because they dont belong to a special class like Arrays, any level of parallelism overcomes the thread overhead and improves performance compared to Serial Old.

## FUTURE WORK

As future work first of all i would like to focus on solving a bug that i have with objects not being processed. This should be a bug of the marking phase but that is yet to be discovered. If i manage to have a fully working version of TeraCache Parallel Old i would like to deploy it on Spark Bench and Renaissance to see the perfomance in real time applications. Moreover i would like to examine possible sources of extra overhead. I would like to break the major gc time down to the times of each phase and also break the total execution time into major gc time, minor gc time and other time. Any optimization to any phase of the major collection will be a huge benefit. Finally i would like to learn a lot more about gc tuning and the parameters of the JVM. By understanding the JVM parameters better i will be able to pick the right ones for every situation and dive more into understanding how to boost the gc performance of applications.

## CONCLUSION

This report focused on presenting a new feature about an existing work called TeraCache. TeraCache is an on heap compute cache designed for Spark. Any big data framework can benefit from it. TeraCache is memory mapped to a fast storage device and is also never garbage collected as a seperate heap than the ordinary Java heap. By avoiding serialization and deserialization overhead and gc overhead the performance improvement than vanilla JVM on Spark is outstanding. The new feature this work adds to TeraCache is that it is imports it to the Parallel Old collector as right now it is able to be deployed only on Serial Old. The highest performance improvement that Parallel Old TC shows against Serial Old TC in a real scenario is 44%. As of right now the only tests that were ran are in microbenchmarks the most anticipated thing for the author is to be able to deploy Parallel Old TC on Spark and see the performance on real time applications. Spark resembles real time scenarios when the heap is stretched regardless of being more weighless because of TeraCache. The Java Heap acts as the execution memory of Spark which nonetheless needs a fair amount of space.

## ACKNOWLEDGMENT

Thanks go to Jack Kolokasis for providing the TeraCache source code in order for this project's contribution to become possible. Moreover thanks to Oracle, RedHat and CARV Laboratory (ICS Forth) for providing the necessary tools such as the OpenJDK platform, the gcc compiler and the hardware.